

Effective Python

Cezary

30. Consider Generators Instead of Returning Lists

- loading all the training data as lists into memory might not work (huge or unbounded dataset)
- generator just holds one batch at a time and discards it after use
- furthermore things like `len()`, `shuffle`, `indexing` are not available since this is an iterator
- anyhow you can create a container class that returns a fresh generator

44. Use Plain Attribute Instead of Setter and Getter Methods

- use simple public attributes
- switch to `@property` when you need behaviour:
 - define a getter with not setter for read-only access
 - keep state in sync (learning rate in optimizer)
 - validation (rejecting bad values)

90. Consider Static Analysis via typing to Obviate Bugs

- install `mypy` and run with `"mypy example.py"`
- static analysis without running the program
- can find hidden bugs as seen in the example

40. Initialize Parent Classes with `super`

- resolve issues with multiple inheritance
- diamond inheritance when subclass inherits from two separate classes that have the same superclass in the hierarchy
- the common superclass's `__init__` method is run multiple times, leading to unexpected behaviour
- `super` function ensures common superclasses in diamond hierarchies are run only once (Method resolution order defines ordering in which superclasses are initialized)
- direct calls ignore the MRO, while `super` doesn't

7. Prefer `enumerate` Over `range`

- if you need items of some sequence and position at the same time
- `enumerate` also works with generators, which have no indexing
- can define with which number you start counting